

Proyek Sistem Operasi

Minggu 3: Inter-Process Communication (IPC) & I/O Redirection

Novalio Daratha
Program Studi Informatika
Universitas Bengkulu

2026

Outline Minggu 3

Mengapa Butuh Inter-Process Communication?

Isolasi Memori Proses

Secara default, sistem operasi mengisolasi ruang alamat (*address space*) setiap proses demi keamanan dan stabilitas. Proses A tidak dapat membaca atau menulis ke memori Proses B secara langsung.

Mengapa Butuh Inter-Process Communication?

Isolasi Memori Proses

Secara default, sistem operasi mengisolasi ruang alamat (*address space*) setiap proses demi keamanan dan stabilitas. Proses A tidak dapat membaca atau menulis ke memori Proses B secara langsung.

Mekanisme IPC pada UNIX

Untuk saling bertukar data dan berkoordinasi, sistem operasi menyediakan mekanisme IPC:

- **Pipes** (Anonim & Named/FIFO)
- **Shared Memory**
- **Message Queues**
- **Sockets** (untuk komunikasi lintas jaringan/host)

UNIX Pipe (pipe())

Konsep Pipe

Saluran komunikasi satu arah (*unidirectional*) dengan prinsip FIFO (First-In, First-Out). Fungsi `pipe(int fd[2])` mengembalikan dua *file descriptor*:

- `fd[0]`: Ujung baca (*read-end*)
- `fd[1]`: Ujung tulis (*write-end*)

Contoh Kode Pembuatan Pipe

```
int fd[2];
if (pipe(fd) == -1) {
    perror("Pipe_gagal_dibuat");
    exit(1);
}
// fd[0] untuk membaca, fd[1] untuk menulis
```

Komunikasi Parent dan Child via Pipe

```
int fd[2];
pipe(fd);
pid_t pid = fork();

if (pid == 0) { // Child: membaca data
    close(fd[1]); // Tutup ujung tulis
    char buffer[100];
    read(fd[0], buffer, sizeof(buffer));
    printf("Child_menerima:_%s\n", buffer);
    close(fd[0]);
} else { // Parent: menulis data
    close(fd[0]); // Tutup ujung baca
    char msg[] = "Halo_dari_Parent!";
    write(fd[1], msg, strlen(msg) + 1);
    close(fd[1]);
    wait(NULL);
}
```

Pengalihan Input/Output: dup2()

System Call dup2()

Menduplikasi *file descriptor* lama ke *file descriptor* baru: `int dup2(int oldfd, int newfd);`

Implementasi Redirection Output (>)

```
// Mengarahkan stdout (fd 1) ke file output.txt
int file_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(file_fd, STDOUT_FILENO);
close(file_fd);

// Sekarang printf atau execvp akan menulis langsung ke file output.txt
printf("Teks_ini_akan_masuk_ke_file_output.txt!\n");
```

Menghubungkan Dua Perintah: `cmd1` | `cmd2`

- Buat sebuah pipe: `pipe(fd)`
- Fork proses untuk `cmd1`: arahkan `stdout` ke `fd[1]` menggunakan `dup2`.
- Fork proses untuk `cmd2`: arahkan `stdin` ke `fd[0]` menggunakan `dup2`.
- Parent menutup kedua `fd` dan menunggu kedua proses selesai.

Tantangan Proyek 1 Minggu 3

Menambahkan penanganan simbol `|`, `<`, `>`, dan `&` (eksekusi *background*) pada program **Custom Shell** Anda!

Ada Pertanyaan?

Praktikum: Buka worksheet3.pdf dan kembangkan fitur pipe pada Custom Shell.